

Capítulo 14

Outros Tópicos sobre C

Exercícios — Resolução Didática com Código C

“Neste documento resolvemos passo a passo os oito exercícios de programação 14.2 a 14.9 do Capítulo 14. Cada solução apresenta o programa em C completo, comentado, seguido de análise da arquitetura adotada, exemplos de execução real e observações didáticas sobre as armadilhas mais comuns. Todos os códigos foram compilados com `gcc -Wall -Wextra -std=c99` e executados sem warnings.”

Convenções adotadas neste documento

- Os códigos estão em caixa cinza-claro com numeração de linha; as saídas de execução, em terminal escuro.
- Cada exercício segue a estrutura (1) enunciado, (2) análise do problema, (3) código em C, (4) execução real, (5) discussão didática.
- As mensagens das interfaces de usuário dos programas estão em português sem acentos, para evitar problemas de codificação de terminais.

1 Exercício 14.2 — Lista de Argumentos de Tamanhos Variáveis

Enunciado. Escreva um programa que calcule o produto de uma série de inteiros passados à função `product` por uma lista de argumentos de tamanhos variáveis. Teste sua função com várias chamadas, cada uma com um número diferente de argumentos.

Análise do problema

A função `product` precisa receber, além dos números a multiplicar, um parâmetro que informe quantos números virão na sequência. Essa é a forma clássica do protocolo de argumentos variáveis em C: o primeiro parâmetro nomeado serve como “meta-informação” sobre o restante.

A assinatura será, então,

```
int product(int count, ...);
```

e o corpo seguirá o protocolo `va_start` → `va_arg` (em *loop*) → `va_end`.

Código em C

```

1  /* Exercício 14.2 - Produto de inteiros via lista variadica */
2  #include <stdio.h>
3  #include <stdarg.h>
4
5  int product(int count, ...);
6
7  int main(void) {
8      printf("Produto de 3 inteiros: %d\n", product(3, 2, 3, 4));
9      printf("Produto de 5 inteiros: %d\n", product(5, 1, 2, 3, 4, 5));
10     printf("Produto de 7 inteiros: %d\n", product(7, 2, 2, 2, 2, 2, 2));
11     printf("Produto de 1 inteiro : %d\n", product(1, 42));
12     printf("Produto de 10 inteiros: %d\n",
13            product(10, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10));
14     return 0;
15 }
16
17 int product(int count, ...) {
18     va_list args;
19     int produto = 1;
20
21     va_start(args, count);
22
23     for (int i = 0; i < count; ++i) {
24         produto *= va_arg(args, int);
25     }
26
27     va_end(args);
28     return produto;
29 }

```

Execução

Saída da execução

```

$ gcc -Wall -Wextra -std=c99 ex14_02.c -o ex14_02
$ ./ex14_02
Produto de 3 inteiros: 24
Produto de 5 inteiros: 120
Produto de 7 inteiros: 128
Produto de 1 inteiro : 42
Produto de 10 inteiros: 3628800

```

Discussão

Conferindo os resultados pela mente: $2 \cdot 3 \cdot 4 = 24$; $5! = 120$; $2^7 = 128$; e $10! = 3,6$ milhões. Tudo bate.

Pontos arquiteturais a destacar:

- O cabeçalho `<stdarg.h>` é indispensável para todo programa que use listas variáveis. Sem ele, o trio `va_list`, `va_start`, `va_arg`, `va_end` fica indefinido.
- `va_start` recebe como segundo argumento o *nome* do último parâmetro nomeado — aqui, `count`. É essa referência que permite à macro localizar o início da região variável na pilha.
- No *loop*, `va_arg(args, int)` extrai o próximo argumento *já interpretando-o como `int`*. Como nosso protocolo só passa inteiros, não há risco — mas se passassem `doubles`, teríamos de pedir `va_arg(args, double)` e o protocolo seria diferente.

- `va_end` fecha a sequência. Em muitas plataformas, ele expande para nada; ainda assim, sua presença é exigida pelo padrão.

Erro Comum de Programação

Não podemos “descobrir” dentro da função quantos argumentos foram passados. `va_arg` sempre avança o “ponteiro lógico” — chamada extra equivale a ler lixo da pilha. O parâmetro `count` é o único mecanismo seguro. (Outra convenção comum, usada por `printf`, é codificar a contagem implicitamente em uma *string* de formato.)

Observação de Engenharia de Software

Listas variáveis transferem para o programador a obrigação de manter a consistência de tipos — algo que normalmente o compilador faria por nós. Por isso, funções com lista variável são *poderosas mas perigosas*. Use-as quando o ganho de expressividade for evidente; do contrário, prefira funções com listas fixas ou funções que recebam arrays.

2 Exercício 14.3 — Impressão de Argumentos da Linha de Comando

Enunciado. Escreva um programa que imprima os argumentos da linha de comando do programa.

Análise do problema

O ponto-chave é a assinatura completa de `main`, na qual `argc` carrega a quantidade e `argv` as strings:

```
int main(int argc, char *argv[])
```

Basta percorrer `argv` com um *loop* de 0 a `argc - 1`.

Código em C

```
1  /* Exercício 14.3 - Imprimir argumentos da linha de comando */
2  #include <stdio.h>
3
4  int main(int argc, char *argv[]) {
5      printf("0 programa recebeu %d argumento(s) da linha de comando:\n\n", argc);
6
7      for (int i = 0; i < argc; ++i) {
8          printf("    argv[%d] = \"%s\"\n", i, argv[i]);
9      }
10
11     return 0;
12 }
```

Execução

Saída da execução

```
$ ./ex14_03 alpha beta gamma 42
0 programa recebeu 5 argumento(s) da linha de comando:

argv[0] = "./ex14_03"
```

```
argv[1] = "alpha"
argv[2] = "beta"
argv[3] = "gamma"
argv[4] = "42"
```

Discussão

Observe pontos sutis dignos de atenção:

- `argv[0]` contém o nome com o qual o programa foi invocado — *não* o nome do arquivo executável necessariamente. Em UNIX, comandos como `exec` podem alterar esse valor; programas como `busybox` usam-no para distinguir entre múltiplos modos de operação.
- O argumento "42" é uma *string*, não um inteiro. Para converter, teríamos de chamar `atoi(argv[i])` ou, melhor ainda, `strtol` (que reporta erros de conversão).
- Os argumentos são separados pela *shell* por espaços em branco — a menos que estejam entre aspas: `./ex14_03 "alpha beta"` produziria apenas 2 elementos.

Boa Prática de Programação

Sempre que escrever um programa que *precisa* de argumentos da linha de comando, verifique `argc` antes de acessar `argv[i]`, com $i \geq 1$. Acessar uma posição inexistente do array é comportamento indefinido. Esta verificação foi feita explicitamente no Exercício 14.4 a seguir.

3 Exercício 14.4 — Classificação com Opção na Linha de Comando

Enunciado. Escreva um programa que classifique um array de inteiros em ordem crescente ou decrescente. O programa deve usar argumentos da linha de comando para passar `-c` (crescente) ou `-d` (decrescente). Nota do livro: este é o formato-padrão para passar opções em UNIX.

Análise do problema

A tarefa combina dois assuntos: (i) ordenação de array (estudada no Capítulo 6, típica do *bubble sort*) e (ii) processamento de `argv` (estudado neste capítulo). A novidade é que a *direção da comparação* se torna paramétrica: passamos uma *flag* booleana para a função de ordenação.

Vamos comparar a *flag* com `strcmp`, da biblioteca de strings (Capítulo 8). Se for inválida, encerramos com mensagem em `stderr` — *boa prática UNIX*.

Código em C

```
1  /* Exercício 14.4 - Ordenacao com flag -c (crescente) ou -d (decrescente) */
2  #include <stdio.h>
3  #include <string.h>
4  #include <stdlib.h>
5
6  #define TAMANHO 10
7
8  void bubbleSort(int v[], int n, int crescente);
9  void imprimirArray(const int v[], int n);
10
11 int main(int argc, char *argv[]) {
```

```

12     int crescente;
13     int dados[TAMANHO] = {37, 12, 89, 4, 56, 23, 71, 8, 45, 99};
14
15     /* Verifica se o flag foi fornecido */
16     if (argc != 2) {
17         fprintf(stderr, "Uso: %s -c | -d\n", argv[0]);
18         fprintf(stderr, "  -c : ordena em ordem crescente\n");
19         fprintf(stderr, "  -d : ordena em ordem decrescente\n");
20         return EXIT_FAILURE;
21     }
22
23     /* Decodifica o flag */
24     if (strcmp(argv[1], "-c") == 0) {
25         crescente = 1;
26     } else if (strcmp(argv[1], "-d") == 0) {
27         crescente = 0;
28     } else {
29         fprintf(stderr, "Opcao desconhecida: \"%s\"\n", argv[1]);
30         return EXIT_FAILURE;
31     }
32
33     printf("Array original:\n");
34     imprimirArray(dados, TAMANHO);
35
36     bubbleSort(dados, TAMANHO, crescente);
37
38     printf("\nArray ordenado em ordem %s:\n",
39           crescente ? "crescente" : "decrescente");
40     imprimirArray(dados, TAMANHO);
41
42     return EXIT_SUCCESS;
43 }
44
45 void bubbleSort(int v[], int n, int crescente) {
46     for (int i = 0; i < n - 1; ++i) {
47         for (int j = 0; j < n - 1 - i; ++j) {
48             int trocar = crescente ? (v[j] > v[j + 1]) : (v[j] < v[j + 1]);
49             if (trocar) {
50                 int aux = v[j];
51                 v[j] = v[j + 1];
52                 v[j + 1] = aux;
53             }
54         }
55     }
56 }
57
58 void imprimirArray(const int v[], int n) {
59     for (int i = 0; i < n; ++i) {
60         printf("%4d", v[i]);
61     }
62     printf("\n");
63 }

```

Execução

Saída da execução

```

$ ./ex14_04 -c
Array original:
 37  12  89   4  56  23  71   8  45  99

```

```

Array ordenado em ordem crescente:
  4  8 12 23 37 45 56 71 89 99

$ ./ex14_04 -d
Array original:
 37 12 89  4 56 23 71  8 45 99

Array ordenado em ordem decrescente:
 99 89 71 56 45 37 23 12  8  4

$ ./ex14_04
Uso: ./ex14_04 -c | -d
    -c : ordena em ordem crescente
    -d : ordena em ordem decrescente

$ ./ex14_04 -x
Opcao desconhecida: "-x"

```

Discussão

- Note que as mensagens de erro vão para `stderr`, não para `stdout`. Isso permite ao usuário redirecionar a saída “útil” para um arquivo sem perder os diagnósticos: `./ex14_04 -c > resultado.txt` continua mostrando os erros na tela.
- O código de saída é `EXIT_FAILURE` (definido em `<stdlib.h>`) quando algo deu errado, e `EXIT_SUCCESS` no caso normal. Essa é a convenção portátil para retornar códigos a *shell scripts* que verifiquem `$?` .
- Para evitar duplicar a função de ordenação (uma para crescente, outra para decrescente), parametrizá-la com uma *flag* é mais elegante. Em projetos maiores, pode-se ir além e passar um *ponteiro para função de comparação* (Capítulo 7) — exatamente o que faz a função `qsort` da biblioteca-padrão.

Observação de Engenharia de Software

A convenção UNIX de *flags* curtas (`-c`, `-d`) seguidas opcionalmente por argumentos teve tanto sucesso que se transformou em padrão POSIX. A biblioteca `<unistd.h>` oferece a função `getopt`, que automatiza esse processamento para programas com muitas opções. Para casos simples como o nosso (uma só *flag*), `strcmp` direto é suficiente.

4 Exercício 14.5 — Arquivos Temporários

Enunciado. Escreva um programa que coloque um espaço entre cada caractere de um arquivo. Primeiro, o programa deve gravar o conteúdo do arquivo em um arquivo temporário, com espaços entre cada caractere; depois copia-o de volta ao arquivo original, sobrescrevendo o conteúdo.

Análise do problema

A grande pergunta arquitetural é: *por que precisamos de um arquivo temporário?* Por que não lê e escrever no mesmo arquivo?

A resposta é simples e instrutiva: se abríssemos o arquivo para leitura e escrita simultaneamente e fossemos modificá-lo *enquanto lemos*, teríamos uma corrida clássica: ao escrever o espaço após o caractere ‘A’, a posição do ‘B’ que viria a seguir mudaria, e podemos acabar lendo o próprio

espaço que acabamos de escrever! O arquivo temporário *quebra a dependência cíclica*: lemos do original para o temporário; quando o original já foi todo lido, abrimo-lo de novo (em modo "w", que trunca), e copiamos do temporário de volta.

A função-chave é `tmpfile()`, que entrega um `FILE *` de uso restrito ao programa em execução — e que será automaticamente removido ao fechar.

Código em C

```

1  /* Exercício 14.5 - Espacar os caracteres de um arquivo usando arquivo temporario
   */
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(int argc, char *argv[]) {
6      if (argc != 2) {
7          fprintf(stderr, "Uso: %s <arquivo>\n", argv[0]);
8          return EXIT_FAILURE;
9      }
10
11     /* Fase 1: abre o arquivo original para leitura e cria um arquivo temporario.
12      Para cada caractere do original, escreve no temporario o caractere
13      seguido de um espaco. */
14     FILE *original = fopen(argv[1], "r");
15     if (original == NULL) {
16         fprintf(stderr, "Erro: nao foi possivel abrir \"%s\" para leitura.\n",
17             argv[1]);
18         return EXIT_FAILURE;
19     }
20
21     FILE *temporario = tmpfile();
22     if (temporario == NULL) {
23         fprintf(stderr, "Erro: nao foi possivel criar arquivo temporario.\n");
24         fclose(original);
25         return EXIT_FAILURE;
26     }
27
28     int c;
29     while ((c = fgetc(original)) != EOF) {
30         fputc(c, temporario);
31         if (c != '\n') {           /* nao colocar espaco apos quebra de linha */
32             fputc(' ', temporario);
33         }
34     }
35     fclose(original);
36
37     /* Fase 2: reabre o original em modo de escrita (truncando-o) e copia
38      todo o conteudo do temporario de volta para ele. */
39     rewind(temporario);
40
41     original = fopen(argv[1], "w");
42     if (original == NULL) {
43         fprintf(stderr, "Erro: nao foi possivel reabrir \"%s\" para escrita.\n",
44             argv[1]);
45         fclose(temporario);
46         return EXIT_FAILURE;
47     }
48
49     while ((c = fgetc(temporario)) != EOF) {
50         fputc(c, original);
51     }
52
53     fclose(original);

```

```
54     fclose(temporario);      /* tmpfile() e auto-removido ao fechar */
55
56     printf("Arquivo \"%s\" reescrito com espacos entre caracteres.\n", argv[1]);
57     return EXIT_SUCCESS;
58 }
```

Execução

Saída da execução

```
$ echo "Ola mundo, programar e divertido!" > teste_orig.txt
$ cat teste_orig.txt
Ola mundo, programar e divertido!

$ ./ex14_05 teste_orig.txt
Arquivo "teste_orig.txt" reescrito com espacos entre caracteres.

$ cat teste_orig.txt
O l a   m u n d o ,   p r o g r a m a r   e   d i v e r t i d o !
```

Discussão

- Optamos por *não* adicionar espaço após ‘\n’, para manter a estrutura de linhas legível. É uma decisão didática que ilustra a importância de tratar separadamente os caracteres de controle.
- Após o *loop* de leitura, não basta `rewind(temporario)`: também precisamos *fechar* o arquivo original e *reabri-lo* em modo "w" para *truncar* seu conteúdo. Se abrísssemos em "r+" apenas, o conteúdo antigo permaneceria nas posições que não fossem sobrescritas.
- O arquivo temporário nunca aparece com nome no sistema de arquivos. Isso oferece duas vantagens: (i) não polui o diretório com arquivos com nomes esquisitos, e (ii) se o programa for interrompido, o sistema operacional ainda assim limpa o temporário quando os *file descriptors* forem fechados.

Dica de Portabilidade

A implementação de `tmpfile()` pode falhar em circunstâncias incomuns — por exemplo, quando o disco está cheio ou quando o sistema atinge o limite de arquivos abertos. Sempre verifique o retorno com `if (temporario == NULL)` e trate a falha. *Programas robustos nunca confiam que alocações de recursos do sistema operacional sempre tenham sucesso.*

Erro Comum de Programação

Se você esquecer o `fclose(original)` antes do reabrir, o *sistema operacional* pode ainda enxergar duas referências ao arquivo, e a operação subsequente pode produzir resultados imprevisíveis (especialmente em Windows). *Feche, reabra, copie* é a sequência segura.

5 Exercício 14.6 — Tratamento de Sinais

Enunciado. Consulte a documentação do compilador para descobrir os sinais aceitos por `<signal.h>`. Escreva um programa com tratadores para `SIGABRT` e `SIGINT`. O programa deve interceptar `SIGABRT` chamando `abort()` e `SIGINT` pela digitação de `<Ctrl>+<C>`.

Análise do problema

Os sinais padrão definidos por `<signal.h>` são seis:

Sinal	Significado
SIGABRT	Encerramento anormal solicitado (via <code>abort()</code>)
SIGFPE	Erro aritmético (divisão por zero, <i>overflow</i>)
SIGILL	Instrução ilegal de máquina
SIGINT	Interrupção interativa (geralmente <code><Ctrl>+<C></code>)
SIGSEGV	Acesso inválido à memória (<i>segmentation fault</i>)
SIGTERM	Pedido de término enviado por outro processo

O nosso programa precisa: (1) *registrar* tratadores para `SIGABRT` e `SIGINT`; (2) aguardar uma ação do usuário; (3) chamar `abort()` para demonstrar a interceptação de `SIGABRT`, ou aceitar `<Ctrl>+<C>` para `SIGINT`.

Código em C

```

1  /* Exercício 14.6 - Tratamento de sinais SIGABRT e SIGINT */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <signal.h>
5
6  void tratadorSinal(int tipoSinal);
7
8  int main(void) {
9      /* Registra o mesmo tratador para os dois sinais */
10     signal(SIGABRT, tratadorSinal);
11     signal(SIGINT,  tratadorSinal);
12
13     printf("Programa iniciado.\n");
14     printf(" - Pressione <Ctrl> + <C> para gerar SIGINT.\n");
15     printf(" - Após 5 segundos sem entrada, chamaremos abort()\n");
16     printf("   para gerar SIGABRT.\n\n");
17
18     /* Aguarda alguma intervencao do usuario; se nada acontecer,
19        dispara abort() para demonstrar a interceptacao de SIGABRT. */
20     printf("Pressione <Enter> para chamar abort() agora,\n");
21     printf("ou <Ctrl>+<C> para gerar SIGINT: ");
22     fflush(stdout);
23
24     int c = getchar();
25     if (c == '\n' || c == EOF) {
26         printf("\nChamando abort()...\n");
27         abort();          /* gera SIGABRT */
28     }
29
30     printf("Programa terminou normalmente.\n");
31     return EXIT_SUCCESS;

```

```

32 }
33
34 void tratadorSinal(int tipoSinal) {
35     const char *nome;
36
37     switch (tipoSinal) {
38         case SIGABRT: nome = "SIGABRT"; break;
39         case SIGINT:  nome = "SIGINT";  break;
40         default:      nome = "desconhecido"; break;
41     }
42
43     printf("\n>>> Sinal interceptado: %s (codigo %d)\n", nome, tipoSinal);
44     printf(">>> O tratador esta encerrando o programa.\n");
45     exit(EXIT_FAILURE);
46 }

```

Execução

Saída da execução

```

$ ./ex14_06
Programa iniciado.
- Pressione <Ctrl> + <C> para gerar SIGINT.
- Apos 5 segundos sem entrada, chamaremos abort()
  para gerar SIGABRT.

Pressione <Enter> para chamar abort() agora,
ou <Ctrl>+<C> para gerar SIGINT: <Enter>

Chamando abort()...

>>> Sinal interceptado: SIGABRT (codigo 6)
>>> O tratador esta encerrando o programa.

$ echo $?
1

```

Discussão

- `signal(SIGABRT, tratadorSinal)` substitui o comportamento padrão do sistema (que seria gerar um *core dump* e encerrar bruscamente) pela chamada da nossa função.
- Note que o mesmo tratador foi registrado para os dois sinais. Dentro dele, o parâmetro `tipoSinal` identifica qual sinal de fato ocorreu — usamos `switch` para diferenciar.
- Em alguns sistemas, ao receber um sinal, o tratador volta automaticamente ao comportamento padrão — precisaria ser *re-registrado* a cada chamada. Em sistemas POSIX modernos, a função mais robusta é `sigaction` (de `<signal.h>`), porém fora do padrão C ANSI estrito.

Erro Comum de Programação

Tratadores de sinais executam em um contexto *assíncrono* extremamente restrito. Em uma visão estrita do padrão C, dentro de um tratador só é seguro: (a) atribuir a variáveis `volatile sig_atomic_t`, (b) chamar funções assíncrono-seguras como `abort`, `_Exit` e `signal` (re-registrar). Usar `printf` dentro de um tratador, como fizemos, é na prática universalmente aceito mas *tecnicamente* indefinido. Para programas de produção, manter o tratador mínimo (apenas atualizar uma *flag*) é a regra de ouro.

Observação de Engenharia de Software

O qualificador `volatile` mencionado na Seção 14.7 do livro encontra exatamente aqui a sua utilidade canonica: variáveis modificadas dentro de tratadores de sinal devem ser declaradas `volatile` para que o compilador não otimize sua leitura, preservando a semântica de alteração assíncrona.

6 Exercício 14.7 — Alocação Dinâmica e Realocação

Enunciado. Escreva um programa que aloque dinamicamente um array de inteiros cujo tamanho é fornecido pelo teclado. Os elementos são lidos do teclado. Imprima o array. Em seguida, realoque-o para metade do tamanho atual. Imprima o array restante para confirmar que corresponde à primeira metade do original.

Análise do problema

A sequência operacional é linear: `malloc` → leitura → impressão → `realloc` → impressão → `free`. O ponto pedagogicamente importante é o tratamento defensivo do retorno de `realloc`: lembre-se da regra de ouro discutida no Exercício 14.1(s): *nunca atribua o retorno de `realloc` diretamente sobre o próprio ponteiro*.

Código em C

```

1  /* Exercício 14.7 - Alocação dinâmica e realloc para metade */
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(void) {
6      int n;
7
8      printf("Digite o tamanho do array (numero par >= 2): ");
9      if (scanf("%d", &n) != 1 || n < 2) {
10         fprintf(stderr, "Tamanho invalido.\n");
11         return EXIT_FAILURE;
12     }
13
14     /* Aloca dinamicamente o array */
15     int *v = malloc(n * sizeof(int));
16     if (v == NULL) {
17         fprintf(stderr, "Erro de alocação.\n");
18         return EXIT_FAILURE;
19     }
20
21     /* Le os elementos */
22     printf("Digite %d inteiros separados por espaços:\n", n);
23     for (int i = 0; i < n; ++i) {
24         if (scanf("%d", &v[i]) != 1) {
25             fprintf(stderr, "Erro de leitura no índice %d.\n", i);
26             free(v);
27             return EXIT_FAILURE;
28         }
29     }
30
31     /* Imprime o array original */
32     printf("\nArray original (%d elementos):\n", n);
33     for (int i = 0; i < n; ++i) {
34         printf("  v[%d] = %d\n", i, v[i]);
35     }
36
37     /* Realoca para metade do tamanho */
38     int novoTamanho = n / 2;
39     int *novo = realloc(v, novoTamanho * sizeof(int));
40     if (novo == NULL) {
41         fprintf(stderr, "Erro de realocação.\n");
42         free(v);
43         return EXIT_FAILURE;

```

```
44     }
45     v = novo;
46
47     /* Imprime o array realocado */
48     printf("\nApos realloc para %d elementos:\n", novoTamanho);
49     for (int i = 0; i < novoTamanho; ++i) {
50         printf("    v[%d] = %d\n", i, v[i]);
51     }
52
53     free(v);
54     return EXIT_SUCCESS;
55 }
```

Execução

Saída da execução

```
$ echo "8
10 20 30 40 50 60 70 80" | ./ex14_07
Digite o tamanho do array (numero par >= 2): Digite 8 inteiros separados por
    espacos:

Array original (8 elementos):
    v[0] = 10
    v[1] = 20
    v[2] = 30
    v[3] = 40
    v[4] = 50
    v[5] = 60
    v[6] = 70
    v[7] = 80

Apos realloc para 4 elementos:
    v[0] = 10
    v[1] = 20
    v[2] = 30
    v[3] = 40
```

Discussão

- Confirmamos empiricamente: após `realloc` para tamanho menor, os `novoTamanho` primeiros elementos são preservados, e o restante é descartado.
- A linha `int *novo = realloc(v, novoTamanho * sizeof(int));` usa um ponteiro auxiliar `novo`. Se `realloc` retornasse `NULL` (falha), `v` ainda apontaria para o bloco original válido e poderíamos liberá-lo com `free(v)`.
- Para realocações que *aumentam* o tamanho, `realloc` pode mover o bloco para outro endereço na *heap*. Por isso é essencial *nunca* guardar ponteiros para o interior de um bloco realocável: eles podem ficar inválidos a qualquer chamada.

Dica de Desempenho

Quando se sabe a priori que um array crescerá, é melhor estratégia alocar uma capacidade inicial generosa e dobrá-la com `realloc` a cada estouro — esse padrão garante custo amortizado constante por inserção. É exatamente assim que os vetores dinâmicos (`std::vector` em C++, `ArrayList` em Java) são implementados.

Boa Prática de Programação

Sempre libere a memória com `free` antes de retornar de `main` (como fizemos), mesmo sabendo que o sistema operacional o faria por você ao terminar o processo. Em programas que viram bibliotecas ou que executam em sistemas embarcados sem SO, esse hábito evita *leaks* graves.

7 Exercício 14.8 — Leitura em Ordem Reversa

Enunciado. Escreva um programa que receba dois argumentos da linha de comando que sejam nomes de arquivos, leia os caracteres do primeiro arquivo um de cada vez, e escreva-os em ordem reversa no segundo arquivo.

Análise do problema

A solução natural usa `fseek` (Capítulo 11) para deslocar a posição de leitura até o fim, descobrir o tamanho com `ftell`, e então iterar de trás para frente, lendo um caractere de cada vez. Não precisamos carregar todo o arquivo para a memória.

Há uma alternativa que *caberia em memória*: `malloc` um buffer do tamanho do arquivo, ler tudo de uma vez com `fread`, e escrever de trás para frente. Para arquivos grandes, porém, isso pode esgotar a RAM. Optamos pela versão com `fseek` por ser sempre segura, ainda que mais lenta no caso de arquivos enormes (uma vez que cada `fseek` é uma chamada ao sistema operacional).

Código em C

```

1  /* Exercício 14.8 - Le um arquivo e escreve seus caracteres em ordem
2     reversa em outro arquivo */
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  int main(int argc, char *argv[]) {
7      if (argc != 3) {
8          fprintf(stderr, "Uso: %s <arquivo_entrada> <arquivo_saida>\n", argv[0]);
9          return EXIT_FAILURE;
10     }
11
12     FILE *entrada = fopen(argv[1], "rb");
13     if (entrada == NULL) {
14         fprintf(stderr, "Erro: nao foi possivel abrir \"%s\".\n", argv[1]);
15         return EXIT_FAILURE;
16     }
17
18     FILE *saida = fopen(argv[2], "wb");
19     if (saida == NULL) {
20         fprintf(stderr, "Erro: nao foi possivel criar \"%s\".\n", argv[2]);
21         fclose(entrada);
22         return EXIT_FAILURE;
23     }
24
25     /* Posiciona-se no final do arquivo para descobrir o tamanho */
26     if (fseek(entrada, 0L, SEEK_END) != 0) {
27         fprintf(stderr, "Erro em fseek.\n");
28         fclose(entrada);
29         fclose(saida);
30         return EXIT_FAILURE;
31     }
32     long tamanho = ftell(entrada);
33
34     /* Le o arquivo um caractere de cada vez, do ultimo para o primeiro */
35     for (long pos = tamanho - 1; pos >= 0; --pos) {
36         if (fseek(entrada, pos, SEEK_SET) != 0) {
37             fprintf(stderr, "Erro em fseek (pos=%ld).\n", pos);
38             break;
39         }
40         int c = fgetc(entrada);
41         if (c == EOF) break;

```

```

42     fputc(c, saida);
43 }
44
45 fclose(entrada);
46 fclose(saida);
47
48 printf("Arquivo \"%s\" gravado com %ld caracteres em ordem reversa "
49       "de \"%s\".\n", argv[2], tamanho, argv[1]);
50 return EXIT_SUCCESS;
51 }

```

Execução

Saída da execução

```

$ printf "abcdefghij" > entrada.txt
$ ./ex14_08 entrada.txt saida.txt
Arquivo "saida.txt" gravado com 10 caracteres em ordem reversa de "entrada.
txt".

$ cat entrada.txt; echo
abcdefghij

$ cat saida.txt; echo
jihgfedcba

```

Discussão

- Abrimos ambos os arquivos em modo binário ("rb" e "wb"). Em sistemas Windows, o modo texto faz conversões automáticas entre `\n` e `\r\n` — o que *quebraria* o nosso cálculo posicional. Em Linux, isso não faz diferença, mas a portabilidade exige cuidado.
- `ftell` retorna a posição atual; após `fseek(..., 0L, SEEK_END)`, esse valor é o tamanho do arquivo em bytes.
- No *loop*, `fseek(entrada, pos, SEEK_SET)` posiciona-se em `pos`, e `fgetc` lê o caractere e avança para `pos+1`. Como decrementamos `pos` a cada iteração, lemos o arquivo de trás para frente.

Dica de Desempenho

O padrão “`fseek + fgetc`” é simples e correto, mas faz duas chamadas ao sistema operacional por byte processado. Para arquivos grandes, a versão com buffer (`malloc + fread + percorrer ao contrário`) é ordens de magnitude mais rápida. Em produção, pode-se ainda usar mapeamento de memória (`mmap` em POSIX) para evitar a cópia entre o núcleo e o programa.

Observação de Engenharia de Software

Generalize a ideia: este é um algoritmo de *processamento sequencial reverso*, padrão útil sempre que precisamos analisar um log do mais recente ao mais antigo, ou avaliar uma expressão em ordem posfixa. A noção subjacente — usar `fseek` para “andar para trás” em um *stream* — será essencial em qualquer manipulação de arquivo de acesso aleatório (Capítulo 11).

8 Exercício 14.9 — goto para Imprimir um Quadrado

Enunciado. Escreva um programa que use comandos `goto` para simular um par de *loops* aninhados, imprimindo um quadrado 5×5 de asteriscos como abaixo. O programa deve usar *apenas* as três instruções `printf` mostradas:

```
printf("*");
printf(" ");
printf("\n");
```

O quadrado a produzir é:

```
*****
*   *
*   *
*   *
*****
```

Análise do problema

A restrição didática é clara: o exercício *não* quer um *loop for* aninhado — ele quer que *simulemos* esse aninhamento *usando apenas goto e rótulos*. É uma forma de mostrar, em pequena escala, como o código de máquina (que é essencialmente *goto*) implementa, de forma trabalhosa, o que linguagens estruturadas oferecem com elegância.

A nossa estrutura será:

- Variáveis `linha` e `coluna`, iniciando em 1.
- Rótulos: `inicioLinha`, `inicioColuna`, `imprimeAsterisco`, `imprimeEspaco`, `avancaColuna`, `fimLinha`, `fim`.
- Em cada (`linha`, `coluna`), decide-se se ali vai um asterisco (borda) ou espaço (interior).

Código em C

```
1  /* Exercício 14.9 - Quadrado de asteriscos usando goto e apenas as tres
2     instrucoes printf permitidas */
3  #include <stdio.h>
4
5  #define LADO 5
6
7  int main(void) {
8     int linha = 1;
9     int coluna;
10
11 inicioLinha:
12     if (linha > LADO) goto fim;
13
14     coluna = 1;
15
16 inicioColuna:
17     if (coluna > LADO) goto fimLinha;
18
19     /* Caractere a imprimir: '*' se estiver na borda; ' ' caso contrario. */
20     if (linha == 1 || linha == LADO || coluna == 1 || coluna == LADO)
21         goto imprimeAsterisco;
22     else
```

```

23     goto imprimeEspaco;
24
25 imprimeAsterisco:
26     printf("*");
27     goto avancaColuna;
28
29 imprimeEspaco:
30     printf(" ");
31     goto avancaColuna;
32
33 avancaColuna:
34     coluna = coluna + 1;
35     goto inicioColuna;
36
37 fimLinha:
38     printf("\n");
39     linha = linha + 1;
40     goto inicioLinha;
41
42 fim:
43     return 0;
44 }

```

Execução

Saída da execução

```

$ ./ex14_09
*****
*      *
*      *
*      *
*****

```

Discussão

- As três instruções `printf` permitidas são exatamente as utilizadas: `printf("*")` (impressão de asterisco), `printf(" ")` (espaço), `printf("\n")` (nova linha).
- A condição de borda é: “a célula (linha, coluna) está na borda se e somente se $linha = 1$ ou $linha = LADO$ ou $coluna = 1$ ou $coluna = LADO$ ”. Isso preenche os quatro lados do quadrado — borda superior, inferior, esquerda, direita —, deixando vazio o miolo.
- Por que duas variáveis e não uma? Poderíamos colapsar (linha, coluna) em um único contador de 1 a 25, mas a ideia é exibir, com clareza, como *loops aninhados* se transformam em `goto`. Cada `goto inicioLinha` é análogo a “continue o *loop* externo”; cada `goto inicioColuna`, ao “continue o *loop* interno”.

Observação de Engenharia de Software

Edsger Dijkstra publicou em 1968 a famosa carta “*Go To Statement Considered Harmful*” (em português: “O comando `goto` é prejudicial”), na qual argumenta que código cheio de `goto` é difícil de entender e de provar correto. Os autores deste livro concordam com Dijkstra: a programação estruturada (com `if`, `while`, `for`, `switch`) torna o fluxo de execução *visualmente correspondente* à estrutura textual do código. Compare a versão com `goto` acima com a versão estruturada equivalente:

```
1  /* Versão estruturada equivalente, para comparacao */
2  for (int linha = 1; linha <= LADO; ++linha) {
3      for (int coluna = 1; coluna <= LADO; ++coluna) {
4          if (linha == 1 || linha == LADO || coluna == 1 || coluna == LADO)
5              printf("*");
6          else
7              printf(" ");
8      }
9      printf("\n");
10 }
```

A versão estruturada é mais curta, mais legível e — crucialmente — expressa diretamente a intenção do programador (“para cada linha, para cada coluna, faça tal coisa”). A versão com `goto` obriga o leitor a reconstruir mentalmente o fluxo a partir dos rótulos. É por isso que, exceto em circunstâncias muito específicas (saída de *loops* profundamente aninhados, tratamento de erros em código de baixo nível), `goto` deve ser evitado.

Boa Prática de Programação

Em código de produção moderno, `goto` ainda *tem* um lugar legítimo: padrão de “*cleanup* em cascata” em código de baixo nível (*kernels*, *drivers*). Veja, por exemplo, o código-fonte do núcleo do Linux: `goto` é frequente, sempre para saltar para um rótulo de limpeza de recursos ao detectar um erro. Fora desse padrão, prefira sempre as construções estruturadas.

Encerramento

Encerramos com este Capítulo 14 a parte procedural de C deste livro — 14 capítulos de viagem desde o primeiro `printf("Bem-vindo a C!")` (Cap. 2) até os recursos mais sofisticados de interação com o sistema operacional, tratamento de erros e gestão de memória dinâmica.

Os oito exercícios resolvidos aqui ilustram, juntos, um princípio profundo: *a vivência plena da linguagem C não está apenas na sintaxe, mas no diálogo com o sistema operacional — através de streams, sinais, argumentos de invocação, alocadores e o sistema de arquivos*. É essa fronteira que distingue a linguagem C de outras — e que faz dela, ainda hoje, o veículo natural para a programação de sistemas.

A partir do Capítulo 15, a obra muda de paradigma: passamos da programação procedural à programação orientada a objetos em C++.